LLM Perception of Time

A Seven-Experiment Scoping Study — Methods, Results, and Reproduction

Time Project (sandbox exploration)

Compiled June 12, 2026

Repository: `explorations/time/` Branch: `main`

Abstract

We ask whether large language models perceive time. The question is widely treated as a single problem, but it conflates three mechanistically unrelated abilities: temporal reasoning in text, temporal self-perception, and time-aware behaviour in an agent. We separate them with seven controlled, API-only experiments (no training) across six current LLMs from two providers, and find that two of the three reduce to surprisingly mechanical explanations.

Self-perception (E1, E2, E5, E6, E10). A model's apparent ability to estimate its own generation latency is, on careful examination, a proxy for its ability to anticipate its own output length. The proxy is well calibrated when latency is driven by visible output (Spearman $\rho = 0.79$), and collapses to no detectable signal when latency is driven by hidden reasoning or by input prefill ($\rho \approx 0$), even though the actual latency varies by a factor of nine or sixteen in those conditions. Estimating in token space beats estimating in second space ($\rho\,0.89$ vs 0.66); the systematic two-fold undershoot in token estimates is mostly closed by a simple in-context self-revision prompt (geometric-mean ratio $0.37 \rightarrow 0.76$) for non-reasoning models, but not for reasoning models. A direct probe of three reasoning models from two providers shows they are not blind to their own reasoning effort — their one-to-ten effort rating tracks actual reasoning-token expenditure with ρ up to 0.91 — they fail only at *magnitude*, and in a model-specific direction (the OpenAI models under-estimate their reasoning, the Anthropic model over-estimates it).

Agency (E3, E4). The same absence of an internal clock means that agents act on elapsed time only when it is supplied explicitly. In a controlled experiment where the transcript text is held fixed and only the elapsed time varies, models adapt their decisions inconsistently. Their correct-decision rate on time-sensitive tasks rises from 0.49 when no time information is present, to 0.80 when timestamps are embedded in the transcript, to 0.96 when the harness injects an explicit elapsed-time field.

Unifying claim. LLMs do not have an internal clock. They have a length estimator, and they respond to elapsed time only when it is handed to them. Headline numbers carry multiplicative intervals (for calibration metrics) and 95% cluster-bootstrap confidence intervals (for inter-experiment statistics). Reproduction details are in Section 2; every number in this document is regenerated by the committed scripts from the committed raw data, with no API access required.

Contents

1 Three lanes, not one

The question "does an LLM perceive time?" is mechanistically ambiguous. A model can be strong at one of the underlying abilities and hopeless at another, and treating them as one question makes the literature confusing. We separate them as follows.

The first lane is *temporal reasoning in text*: ordering events, arithmetic on dates, reasoning about durations stated in language. This is studied extensively, with well-known benchmarks such as TimeBench (Chu et al., 2024), TRAM (Wang and Zhao, 2024), Test of Time (Fatemi et al., 2025), and TIME (Wei et al., 2025). Performance is largely a matter of in-context reasoning and is, in our view, saturated as a research direction. We do not pursue it.

The second lane is *temporal self-perception*: can a model estimate the time or length of its *own* computation, before or after that computation has happened? The leading negative result here is Garikaparthi (2026), who reports that LLMs overshoot their own task-duration estimates by a factor of four to seven and fail relative ordering at chance. A complementary line of work argues that some self-perception of duration is possible if the model can map token counts to wall-clock time (Wang et al., 2025). The two findings appear to conflict and we resolve the conflict mechanistically in this report. Self-perception is the subject of E1, E2, E5, E6, and E10.

The third lane is *time-aware agency*: does an agent change its tool-use or decision behaviour based on elapsed time embedded in its transcript? The leading work here is the TicToc benchmark (Cheng et al., 2025), which finds that agents fail to adapt tool-use decisions to elapsed time and that simply inserting timestamps in the prompt helps only marginally (about five percentage points). The authors conclude that post-training is needed. We test an alternative hypothesis — that the deficit is partly a missing-sensor problem addressable at the harness layer — in E3 and E4.

Supporting documents in the repository: `research-notes.md` (literature notes and reading list), `experiments.md` (the full E1–E10 plan and status), `SUMMARY.md` (a shorter prose synthesis), and `direction-train-it-in.md` (an alternative research direction that treats E4's sensor result as a ceiling and asks how much a model could internalise through post-training rather than read off the sensor).

2 Shared infrastructure and reproducibility

This section is a complete description of how the experiments are run and how to reproduce every number in this report. The implementation lives in `common.py` and is imported by every experiment.

2.1 Model roster

Every experiment draws from one fixed roster, mapped from a short "roster key" used throughout the code to the provider's API model identifier. The study was run on 2026-06-11.

Roster key	Provider	API model identifier	Reasoning engaged?
`haiku`	Anthropic	`claude-haiku-4-5-20251001`	no (off)
`sonnet`	Anthropic	`claude-sonnet-4-6`	no (off)
`opus`	Anthropic	`claude-opus-4-8`	no (off)
`fable`	Anthropic	`claude-fable-5`	yes (adaptive)[†]
`gpt4o-mini`	OpenAI	`gpt-4o-mini`	no
`gpt4o`	OpenAI	`gpt-4o`	no
`gpt5`	OpenAI	`gpt-5`	yes
`gpt5.2`	OpenAI	`gpt-5.2`	yes
`o4-mini`	OpenAI	`o4-mini`	yes

[†]fable is used only in E10 (Section 9), as a cross-provider reasoning model. Its thinking is forced-adaptive: it cannot be disabled and has no budget control, and `temperature` is deprecated for it.

Reasoning capability versus reasoning engaged — an important clarification. The last column records whether the model *actually performed hidden reasoning during our runs*, not whether it is reasoning-*capable*. The distinction matters and we were careful to verify it from the code, because it affects how every self-perception result should be read.

The three Anthropic models (haiku, sonnet, opus) are all reasoning-capable: they support extended thinking. We disabled it *explicitly* rather than relying on a default. The single call site in `common.py` passes `thinking={"type": "disabled"}` on every Anthropic request, and records the thinking-token count returned in the usage object, which is zero on every Anthropic row in the data. (An earlier version of the harness left thinking implicit; we made it explicit precisely so that no result depends on an undocumented model default.) With thinking disabled, on a hard problem (checking primality, computing a derangement of nine) these models return a single `text` content block, zero thinking tokens, and the answer directly, with no chain of thought. Every haiku, sonnet, and opus data point in this report is therefore a *thinking-off* run, verifiable from the recorded `reasoning_tokens` field. The three OpenAI models marked "yes" (gpt5, gpt5.2, o4-mini) did perform hidden reasoning, which the API reports as nonzero `reasoning_tokens`; for them, wall-clock latency decouples from the length of the visible reply. This decoupling is the recurring boundary that E10 (Section 9) probes directly.

Consequence for interpretation. The "self-perception is a length proxy" finding (E1, E2, E5) is established for reasoning-capable models *run with thinking off*. We should not assume it holds when their thinking is engaged: with extended thinking on, an Anthropic model's latency would include hidden thinking tokens, and we would expect it to undershoot its own latency in the same way gpt5.2 and o4-mini did. Testing the haiku/sonnet/opus models with thinking enabled is a clean follow-up that we have not run; note that the documented `thinking={type:enabled}` parameter is rejected by the API gateway in this environment for these models, so engaging thinking would require a different mechanism. We did, however, add one reasoning model that thinks by default: Fable 5 (`claude-fable-5`), whose thinking is forced-adaptive (it cannot be disabled). Fable is used only in E10 (Section 9), where it serves as an Anthropic cross-provider counterpart to the OpenAI reasoning models.

The roster has two provider quirks that `common.py` handles transparently. First, the OpenAI reasoning models require the `max_completion_tokens` field rather than the `max_tokens` field used by all other models, and they reject any `temperature` argument. Second, `claude-opus-4-8` also rejects `temperature` in this harness, so E4 passes `temperature=None` for opus. The token budget for reasoning models is bumped to at least 4096 because hidden reasoning consumes that budget

before any visible output is generated.

2.2 The call interface

Every experiment imports a single helper so that provider differences are handled in one place:

```
from common import call
r = call("sonnet", prompt, max_tokens=1024)
# r is a Result(
# model, text, latency_s,
# input_tokens, output_tokens, reasoning_tokens,
# ok, error
# )
```

The `latency_s` field is wall-clock time measured by Python's `time.perf_counter()` around the network call. It therefore includes network round-trip and queueing time as well as generation time. The `reasoning_tokens` field is populated for OpenAI reasoning models (zero otherwise) and is read from the API response's `completion_tokens_details.reasoning_tokens`. Transient errors are retried with exponential backoff (up to four attempts).

2.3 Environment

We use a Python 3.11 virtualenv at `explorations/time/.venv`. The exact dependency versions used during the study are pinned in `requirements.txt`: `anthropic 0.109.1`, `openai 2.41.1`, `numpy 2.4.6`, `pandas 3.0.3`, `scipy 1.17.1`, and `matplotlib 3.10.9`. Secrets are read from the environment variables `ANTHROPIC_API_KEY` and `OPENAI_API_KEY`.

2.4 Data format and the resume/dedup scheme

Each experiment directory `eN-*/` contains, at minimum, `run.py` (the data collection script), `analyze.py` (the statistics-and-figures script), a `results.jsonl` file with one JSON record per API call, a `REPORT.md` with the self-contained per-experiment writeup, and PNG figures generated by `analyze.py`.

Data collection is append-only. `run.py` is idempotent in the following sense: at startup it reads any existing `results.jsonl` and builds a set of logical keys (typically the tuple of model, task identifier, trial number, and condition) that are already present, then skips those at execution time. An interrupted run therefore resumes from where it stopped without re-spending API budget on completed cells.

This scheme has a known hazard that we hit during the study and document here. The `load_done()` check at startup is not safe against concurrent writers: if two `run.py` processes for the same experiment are alive simultaneously, both load the same "already done" set and write to the same file, producing duplicate rows. This happened to E4 during the study, producing 492 duplicate rows that were silently filed before the bug was noticed. The fix is the script `dedup.py`, which canonicalises a `results.jsonl` by its logical key (keeping the last occurrence of each key) before analysis. We recommend running `dedup.py` as a precaution before `analyze.py`, although `analyze.py` itself is tolerant of duplicate rows. A proper fix would be a file lock or a single coordinating process; we have not implemented one because the dedup workflow is sufficient for the studied scale.

2.5 Metrics

We use four statistics throughout the report. Three (Spearman ρ, ordering accuracy, correct-decision rate) are standard; the geometric-mean ratio is less standard outside calibration work and we motivate it here.

Spearman rank correlation. For paired observations $\{(y_i, \hat{y}_i)\}$, Spearman's ρ is Pearson's correlation on the ranks of y and $\hat{y}$. It measures *ordering quality*: $\rho = 1$ means the predicted and actual orderings agree perfectly, $\rho = 0$ means they are unrelated, $\rho = -1$ means they are opposite. We use ρ rather than Pearson's r because most of the predicted quantities in this study are positive and span orders of magnitude, where a non-parametric rank measure is more robust than one that assumes a linear relationship.

Ordering accuracy. The fraction of task pairs (a, b) for which the predicted ranking agrees with the actual ranking, with ties excluded. Chance is 0.5. This is a finite-sample, interpretable companion to ρ that does not require any distributional assumption.

Geometric-mean ratio. For predicted-vs-actual data where both quantities are strictly positive, let $r_i = \hat{y}_i / y_i$. The geometric-mean ratio is

$$\mathrm{GM} \;=\; \exp\Big(\tfrac{1}{n}\sum_i \ln r_i\Big) \;=\; \Big(\textstyle\prod_i r_i\Big)^{1/n}.$$

It equals 1 when predictions are unbiased on a multiplicative scale, exceeds 1 for systematic overshoots, and lies below 1 for systematic undershoots. We use the GM ratio rather than mean squared error (MSE) for two reasons. First, the quantities we are calibrating — latency in seconds, output length in tokens, hidden reasoning tokens — span two to three orders of magnitude. Squared additive error in that regime is dominated by the longest tasks: a fifty-percent overshoot on a 600-token essay (squared error $300^2 = 90{,}000$ tok^2) would swamp the same fifty-percent overshoot on a one-word reply ($0.5^2 = 0.25$ tok^2), so MSE would measure the essay tasks and ignore the rest. The GM ratio treats both as equally miscalibrated by a factor of 1.5. Second, the GM ratio cleanly separates a model's *systematic bias* (the central value of r) from its *ranking quality* (Spearman ρ); MSE conflates the two into a single number. Together, GM and ρ give a two-dimensional picture: GM near 1 with a high ρ is calibrated, GM far from 1 with a high ρ is biased but consistent (a fixed scaling correction would fix it), and a low ρ means the predictor has no ranking signal at all.

Multiplicative interval. The GM ratio is a central tendency on the log scale, so its natural notion of spread is also multiplicative. We define the *geometric standard deviation* of the ratios as

$$s_g \;=\; \exp\Big(\mathrm{sd}_i \ln r_i\Big),$$

which is a dimensionless multiplicative factor. We then report the symmetric multiplicative interval $[\mathrm{GM}/s_g,\ \mathrm{GM} \cdot s_g]$, which contains roughly the middle 68% of trial-level ratios under a log-normal model. This is the log-space analogue of "mean ± 1 standard deviation." We display it as a subscript next to the GM ratio. The notation $0.93_{\,[0.55,\,1.58]}$ means GM $= 0.93$ with a multiplicative spread of approximately $1.7\times$ around it. The interval is asymmetric in additive terms (the upper-edge gap is $1.58 - 0.93 = 0.65$, larger than the lower-edge gap of $0.93 - 0.55 = 0.38$) but symmetric in multiplicative terms (both endpoints differ from 0.93 by the same factor $s_g \approx 1.70$). All multiplicative intervals in this report are produced by `gm_intervals.py` and are reproducible directly from the committed `results.jsonl` files.

Correct-decision rate. For experiments where each cell yields a labeled decision (E3, E4), we report the simple fraction of decisions whose label matches a pre-specified correct label. Confidence intervals for these rates are produced by cluster bootstrap (see Section 10) rather than by a multiplicative interval, because the quantity is a proportion rather than a positive ratio.

2.6 How to reproduce

From `explorations/time/`:

```
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
export ANTHROPIC_API_KEY=... OPENAI_API_KEY=...

# Re-derive every table and figure offline from the committed raw data
# (no API calls, no API budget):
for d in e1-self-duration e2-token-time e3-log-gap-probe \
        e4-harness-clock e5-latency-decoupled e6-length-bias \
        e10-reasoning-tokens; do
  ( cd $d && python analyze.py )
done

# Multiplicative intervals reported in every table:
python gm_intervals.py

# 95% cluster-bootstrap CIs (Section~\ref{sec:ci}):
python bootstrap_ci.py

# To re-collect raw data (spends API budget). run.py RESUMES from results.jsonl,
# so for a true from-scratch run, move the existing file aside first:
# cd eN-*
# mv results.jsonl results.jsonl.bak
# python run.py
# python ../dedup.py eN-* # only needed if writers ran concurrently
# Quick smoke test of one model/task without a full sweep:
# python run.py dryrun
# Collection knobs (model ROSTER, N_TRIALS) live at the top of each run.py.
```

Re-running `analyze.py` on the committed `results.jsonl` files reproduces every statistic in this report exactly. Re-running `run.py` to collect fresh data reproduces them up to sampling noise.

2.7 Determinism caveats

Calls use the provider's default temperature; we do not pin temperature to zero because (a) several models in the roster reject the parameter, and (b) we want the trial-to-trial variation to surface as part of the analysis. As a consequence, re-collected data will vary from trial to trial. Absolute wall-clock numbers are harness- and hardware-specific. We therefore lead with rank statistics (Spearman ρ, ordering accuracy) and between-condition contrasts, which are robust, and report absolute latencies as distributions over multiple trials rather than as point estimates. The empirically observed trial-to-trial coefficient of variation of latency is reported per experiment.

3 E1 — Self-duration calibration

Question. Given a task, can a model estimate, in seconds, how long it will take to generate its response? This is the basic question of *temporal self-perception*.

Hypothesis (from the literature). No. Garikaparthi (2026) reports that LLMs systematically overshoot their own task-duration estimates by a factor of four to seven, and that their relative ordering of task durations is at or below chance. The expected outcome, against that hypothesis, is a GM ratio in the range 4 to 7 and a Spearman ρ close to zero.

Setup. We use twelve tasks designed to span a wide range of expected output lengths, from a one-word answer ("Answer in one word: what is the capital of France?") to roughly a five-hundred-word essay ("Write a 500-word essay on the importance of curiosity in science."). The full list of task identifiers, prompts, and the rationale for each is given in Appendix D.1. For each combination of model, task, and trial number (with three trials per cell), we perform three API calls per trial in the following order:

1. **Pre-estimate.** A wrapped prompt that quotes the upcoming task and asks the model to predict, as a single number, how many seconds it will take to generate its full reply. The exact wrapper is in Appendix E.

2. **Generation.** The task prompt is sent verbatim. We record the wall-clock latency `latency_s` (using `time.perf_counter()` around the API call) and the output length `output_tokens` (from the API response's usage field).

3. **Post-estimate.** A fresh call shows the model the task it was given and the response it actually produced, and asks how long that generation took. This isolates *post-hoc recall* from *pre-hoc prediction*.

Six models from the roster (see Section 2.1) are used: `haiku`, `sonnet`, `opus`, `gpt4o-mini`, `gpt4o`, and `gpt5.2`. Total: 648 API calls, organised into 216 (model, task, trial) units.

Results. The hypothesis from the literature is not supported in this task set. Table 1 reports the per-model results. Overall, the geometric-mean ratio of predicted-over-actual latency is $0.81_{\,[0.31,\,2.14]}$, which is mildly biased downward (predictions slightly undershoot) rather than overshooting by a factor of four to seven. The overall Spearman rank correlation is 0.75 (95% bootstrap CI $[0.62, 0.84]$, $p < 0.001$), not zero. Per-model ordering accuracy ranges from 0.79 (gpt5.2) to 0.95 (sonnet), well above the chance level of 0.5.

Table 1: E1: per-model calibration. The GM ratio is the geometric mean of (pre-estimate seconds) / (actual latency seconds). Subscripts give the multiplicative interval $[\mathrm{GM}/s_g,\ \mathrm{GM}\cdot s_g]$ as defined in Section 2.5. The order accuracy is the fraction of task pairs whose predicted order matches the actual order (chance = 0.5).

model	GM ratio (pre)	ρ_{pre}	order acc.	ρ_{post}
opus	$0.93_{\,[0.55,\,1.58]}$	0.94	0.94	0.95
sonnet	$0.76_{\,[0.48,\,1.20]}$	0.85	0.95	0.85
haiku	$0.86_{\,[0.59,\,1.27]}$	0.89	0.88	0.76
gpt4o	$1.04_{\,[0.20,\,5.51]}$	0.83	0.86	0.77
gpt4o-mini	$1.36_{\,[0.64,\,2.87]}$	0.74	0.86	0.89
gpt5.2	$0.32_{\,[0.15,\,0.69]}$	0.74	0.79	0.84
pooled	$0.81_{\,[0.31,\,2.14]}$	**0.75**	—	—

The multiplicative intervals in Table 1 are informative beyond the central values. Anthropic models have tight spread (factors of 1.5–1.7× around the central GM ratio), meaning their per-trial predictions are not only on average correct but also individually close. The two OpenAI non-reasoning models are wider; gpt4o in particular has a 5.3× spread, meaning individual ratios scatter widely even though the central value is near 1. The OpenAI reasoning model gpt5.2 is the

only model with a central tendency far from 1: it systematically *undershoots* its own latency by roughly three-fold, and its spread covers no part of the calibrated region $[0.7, 1.4]$. We return to this in Section 3.1 below.

Per-model scatter plots of predicted vs. actual latency are shown in Figure 1. Pooling models in a single panel (as the original draft of this report did) obscured the structure: each model has its own distinct shape, and the figure is much more legible split per-model.

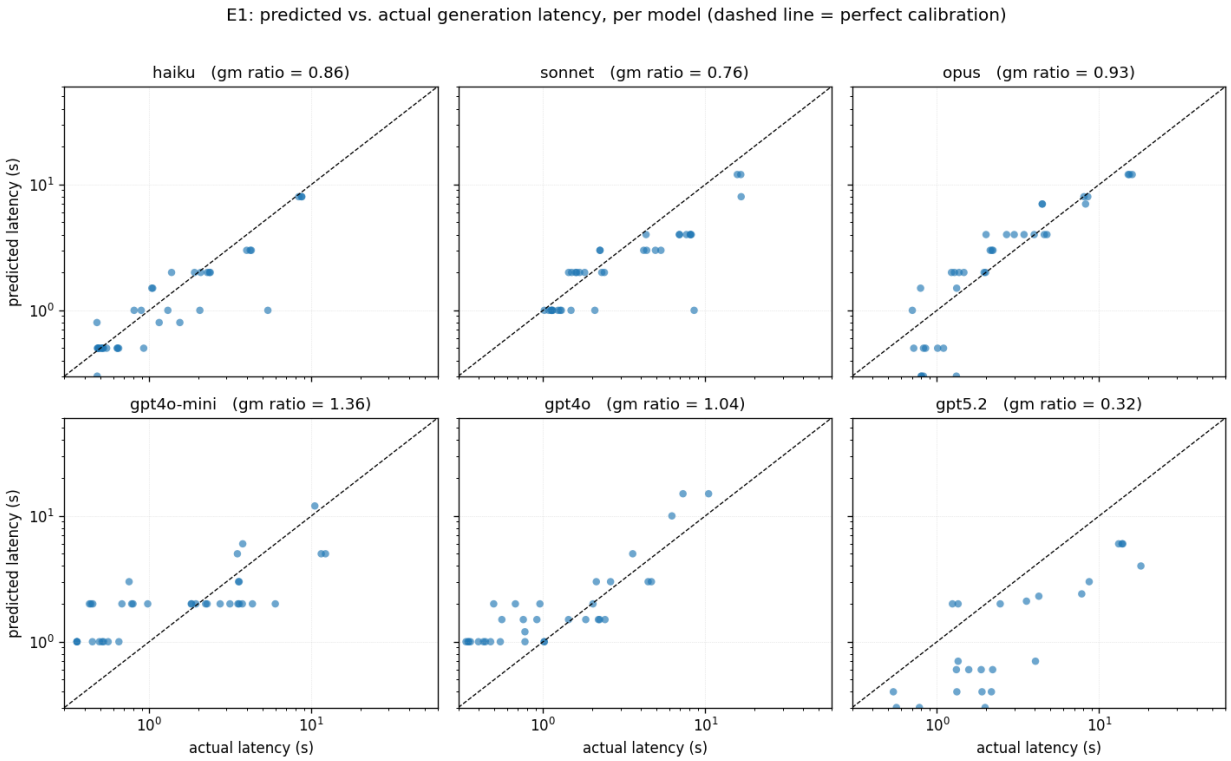

Figure 1: E1: predicted vs. actual generation latency per model, on log–log axes. Each point is one (task, trial) cell. The dashed line is the identity $\hat{y} = y$. The annotation in each panel is the per-model GM ratio of predicted-over-actual latency. Anthropic models cluster tightly around the diagonal; gpt4o has a near-diagonal central tendency but wide scatter; gpt5.2 (a reasoning model) sits systematically below the diagonal, undershooting its own latency by roughly three-fold.

The empirically observed trial-to-trial coefficient of variation of latency (the noise floor) is small relative to the calibration signal: 0.04 for haiku, 0.06 for sonnet, 0.14 for opus, 0.07 for gpt4o-mini, 0.11 for gpt4o, 0.11 for gpt5.2 (median values across tasks). The calibration signal we observe is therefore not an artifact of noisy latency measurements.

3.1 Why the result contradicts the literature, mechanistically

Observation. On this task set, five of six models predict their own generation latency competently (Table 1), in contrast to the published claim of a four-to-seven fold overshoot and chance-level ordering. The exception is the reasoning model gpt5.2, which systematically *undershoots* (GM 0.32, interval $[0.15, 0.69]$).

Interpretation. The task set varies tasks primarily by *expected output length*: one-word answers versus paragraphs versus essays. For non-reasoning models, wall-clock latency is approximately a linear function of output token count, so a model that can anticipate roughly how long its own output will be can also predict its own latency reasonably well — not because it has a clock, but because output length is the thing that drives the clock in this regime. Two pieces of evidence support this length-proxy reading:

- The reasoning model gpt5.2 is the worst-calibrated. Its latency is dominated by hidden reasoning tokens (the API's `reasoning_tokens` field), which are not present in the visible output it can anticipate. The length proxy is broken precisely where output length stops driving latency, and gpt5.2 undershoots accordingly.
- Inspection of individual cells shows the systematic undershoots clustering on free-form generations (the `code` and `story` tasks), where the model underestimates its own verbosity, rather than on short factual answers, where output length is bounded by the question.

Generalisation. The literature's negative result (Garikaparthi, 2026) is very likely correct in a different regime: tasks where latency varies for reasons the model cannot infer from its own anticipated output, such as variable tool-latency, reasoning depth, hardware differences, or batching. Where our task set strips those factors out and lets output length drive latency, the apparent calibration largely returns. E5 (Section 7) tests this reading directly by decoupling latency from output length and verifying that the calibration collapses.

Verdict. Rejected for this regime, with a clean mechanistic explanation: models predict their own latency well when output length drives it, via an implicit length proxy rather than via a clock; the proxy fails for reasoning models whose latency hides in tokens they cannot see.

4 E2 — Token/step proxy versus second-space

Question. If E1's apparent calibration is in fact a length-anticipation proxy, then asking the model to predict in *token space* (the quantity it actually has access to) should be better-calibrated and more consistent across models than asking it to predict in *second space* (a quantity it has no direct access to).

Hypothesis. Yes. Token-space estimates should beat second-space estimates on both ordering (ρ) and bias (GM ratio).

Setup. We use a task set similar in spirit to E1's, with twelve tasks spanning short to long expected outputs (full list in Appendix D.2). For each (model, task, trial) combination, we run four estimation conditions and one real generation, in separate API calls:

- **(a) seconds.** Predict, in seconds, how long the response will take.
- **(b) tokens.** Predict, in output tokens, how long the response will be.
- **(c) tokens converted to seconds.** Take the token estimate from (b) and divide by an empirically measured per-model tokens-per-second rate (computed from the generations themselves). This is not a separate API call.

- **(d) reason-then-seconds.** Predict in seconds, but first instructed to think briefly about the response length before producing the number.

The full prompt wrappers are in Appendix E. Six models are used: `haiku`, `sonnet`, `opus`, `gpt4o-mini`, `gpt4o`, and `o4-mini`. Total: 864 API calls, with zero parse failures across all conditions.

Results. Table 2 reports the pooled (across model and task) calibration for each condition. Token-space estimates have a Spearman ρ of 0.89 against actual output tokens, compared to 0.66 for second-space estimates against actual latency, a difference of $\Delta\rho = +0.23$. Token-space estimates are also much more consistent across models (per-model ρ in the range 0.90–0.99 for non-reasoning models), where second-space estimates have per-model GM ratios ranging from 0.51 (o4-mini, severe undershoot) to 6.6 (gpt4o-mini, severe overshoot). The cross-model variation in scale destroys the pooled second-space correlation; token space normalises it.

Table 2: E2: pooled calibration by estimation condition. Multiplicative intervals as in Table 1.

condition	ρ (predicted vs. actual)	GM ratio (predicted/actual)
(a) seconds	0.657	$2.16_{\,[0.76,\,6.18]}$
(b) tokens	**0.889**	$0.49_{\,[0.18,\,1.33]}$
(c) tokens→seconds	0.829	(0.24, derived)
(d) reason-then-sec	0.545	$1.08_{\,[0.28,\,4.15]}$

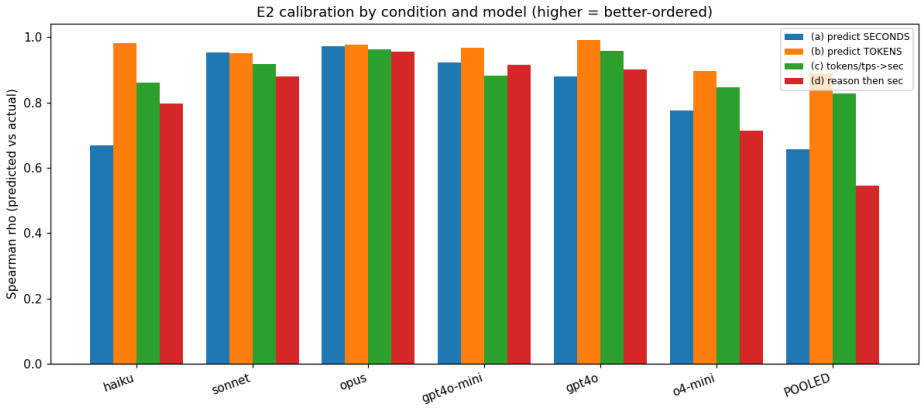

Figure 2: E2: Spearman rank correlation of estimate vs. actual, by estimation condition and model. Token-space (b) is uniformly high across non-reasoning models; second-space (a) is lower and more variable; reasoning-then-seconds (d) does not help.

Three interpretive points are worth drawing out.

First, second-space estimates correlate well *within* a model but have wildly model-specific scale errors: gpt4o-mini overshoots by 6.6×, gpt4o by 5.2×, o4-mini undershoots by 0.51×. When we pool models, those scale errors destroy the rank correlation (pooled ρ drops to 0.66). Asking in seconds requires the model to know not only how long its output will be but also how fast its own (hardware-dependent, batch-dependent) inference is, and the latter is information it does not have.

Second, asking in tokens normalises this: the pooled ρ rises to 0.89, every non-reasoning model has per-model $\rho \geq 0.90$, and the per-model GM ratios cluster around 0.5–0.8. Models systematically

under-predict their own output token count by roughly a factor of two, but they do so consistently. A bias correction is easy; the ordering is intact.

Third, converting token estimates to seconds via a measured tokens-per-second constant (c) compounds the two-fold token undershoot — the GM ratio drops to 0.24 — but preserves the ρ at 0.83. A token-to-time pipeline therefore needs a bias correction, not just a raw constant.

Fourth, asking the model to reason about response size before answering in seconds (d) does not help and slightly hurts pooled ρ (from 0.66 to 0.55), because it reintroduces the per-model scale errors. Deliberation is not calibration.

The OpenAI reasoning model o4-mini is the outlier throughout. Its token-space GM ratio is 0.10, meaning it estimates its own output to be roughly ten times shorter than it actually is. This is consistent with the E1 finding for gpt5.2: reasoning models cannot count tokens they will spend on hidden reasoning, and visible output length is a poor proxy for their total token output. E10 (Section 9) probes this directly. The per-run scatter of predicted vs. actual output tokens (one dot per trial) is Figure 8 in Appendix C.

Verdict. Supported. Models track their own output length, not time. The natural target for any self-estimation procedure is token (or step) space, with a learned bias correction applied before conversion to seconds. The approach fails precisely where output length stops predicting latency — reasoning models with hidden tokens, the same boundary E1 found and E5 will confirm.

5 E3 — Log-gap probe: are agents timestamp-blind?

Question. If an agent's transcript carries timestamps that imply substantial elapsed time, does the agent's downstream decision behaviour respond to that elapsed time?

Hypothesis (from the literature). Largely no. Cheng et al. (2025) show on the TicToc benchmark that LLMs fail to adapt tool-use decisions to elapsed time, and that inserting timestamps in the prompt helps only marginally (about five percentage points). We test this with a cleaner single-variable manipulation: hold the transcript text completely fixed, vary only the elapsed time encoded in the timestamps.

Setup. We construct nine short, time-sensitive scenarios. Each scenario contains an action recorded at some time t_0 followed by a user request at $t_0 + \text{gap}$, and the model must decide between a "fresh" label (re-use the cached information, treat the session as valid, etc.) and a "stale" label (refetch, expire, wait). Each scenario declares a human-sensible staleness threshold; gaps shorter than the threshold should produce the fresh label, gaps longer than the threshold should produce the stale label. The nine scenarios cover the families summarised in Table 3; the full prompts are in Appendix D.3.

For each scenario, the transcript text is held strictly fixed and only the elapsed gap varies over the ladder {1s, 1min, 1h, 1day, 1week}. A no-timestamp control with all time information stripped is also run, against which the timed conditions are compared. Decisions are forced to a single-word label (e.g. "REUSE" or "REFETCH") and parsed by exact match. Five models are used: `haiku`, `sonnet`, `opus`, `gpt4o`, and the reasoning model `gpt5.2`. Three trials per cell. Total: 810 API calls.

We deliberately separate two distinct metrics, because they correspond to different failure modes:

Sensitivity. For a given scenario and model, does the decision label change *at all* across the gap ladder? A model that always answers "REUSE" regardless of gap has sensitivity zero on that

Table 3: E3: scenario summary. The threshold is the gap above which the correct decision flips from fresh to stale. Each scenario has a transcript that records a prior action at t_0 and asks a decision question at $t_0 + \text{gap}$.

scenario id	decision (fresh / stale)	threshold
`stock_price`	REUSE / REFETCH	1 minute
`weather`	REUSE / REFETCH	3 hours
`inventory`	REUSE / REFETCH	1 day
`twofa_code`	ACCEPT / REJECT	10 minutes
`session_token`	VALID / EXPIRED	1 hour
`wait_a_bit`	WAIT / CHECK	1 minute
`geocode`	REUSE / REFETCH	∼2 years (effectively never stale)
`med_dose`	WAIT / DOSE	4 hours
`food_safety`	SAFE / DISCARD	2 hours

scenario — it is not reading the clock. Reported as the fraction of scenarios on which the decision changes.

Correctness. For the timed conditions, what fraction of (scenario, gap) cells produce the decision that matches the scenario's human-sensible threshold? A model that flips at the wrong gap is sensitive but uncalibrated.

Results. The blunt "timestamp-blind" framing is not supported by our data; instead, sensitivity is *graded* by model strength and correctness is uniformly good *when* the model is sensitive at all (Table 4). The strongest model, opus, responds to the gap in 9 of 9 scenarios; the weakest, haiku, in only 5 of 9. When models do respond, their correctness ranges from 0.80 to 0.93.

Table 4: E3: sensitivity and correctness by model.

model	sensitivity	correctness
opus	1.00	0.86
sonnet	0.78	0.93
gpt5.2	0.78	0.89
gpt4o	0.67	0.80
haiku	0.56	0.82

The control condition (no timestamps) changes the decision for no model except opus (33% of scenarios), confirming that the timed conditions are not just measuring base-rate label bias.

Interpretation. The deficit is *inconsistent attention to the clock* (a sensitivity problem) rather than *poor temporal judgement* (a correctness problem). The literature finding of "temporal blindness" (Cheng et al., 2025) is real but, on a clean single-variable manipulation, it is graded by model strength rather than uniform across the population. Opus's 9/9 sensitivity is notable: at the strongest tier, the apparent blindness essentially disappears under this cleaner manipulation.

Verdict. The hypothesis is partially supported and the framing is refined. Strong models do read the clock, and read it correctly when they read it. Weak models flatten, particularly on extreme

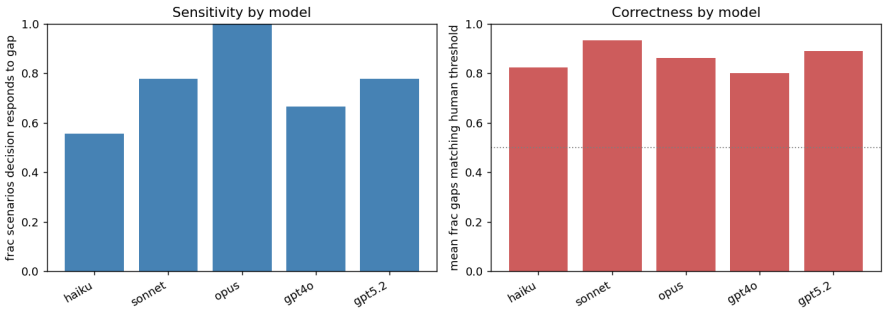

Figure 3: E3: per-scenario sensitivity by model. A black cell indicates the model's decision changed across the gap ladder for that scenario. opus reads the clock in every scenario; weaker models flatten to a constant decision on extreme-threshold scenarios (`stock_price` at one minute, `geocode` at two years).

thresholds (the second-scale `stock_price` and the never-stale `geocode`). The natural extension is to test whether providing the elapsed time *explicitly* (as a structured field rather than as embedded timestamps the model must compute against) lifts the weak models; that is E4.

6 E4 — Harness clock: sensor versus text timestamps

Question. How much of agent "temporal blindness" is a model-capability deficit, and how much is a missing-sensor problem that could be fixed at the harness layer without any training?

Hypothesis. If we explicitly inject an elapsed-time field into the prompt — a plainly-labelled, structured signal of how much time has passed since the prior action — the correct-decision rate should rise above what the model achieves when timestamps are embedded in transcript text and the model has to compute the gap itself.

Setup. We construct nine time-sensitive scenarios, similar in spirit to E3 but with thresholds and gap ladders that are scenario-specific (so that each scenario has gaps both clearly below and clearly above its staleness threshold). The scenario families are summarised in Table 5; the full prompts and condition wrappers are in Appendix D.4.

For each (model, scenario, gap) combination, we run three presentation conditions:

(a) no time. The transcript carries no temporal information at all. Used as a baseline.

(b) text timestamps. The transcript contains absolute timestamps on each line (e.g. "[2026-06-11 09:00:00] tool fetch_price..."). The model must compute the elapsed gap itself.

(c) harness clock. The transcript is timestamp-free, but a structured field "`[elapsed since last action: 3 hours 12 minutes]`" is injected immediately before the decision question. The model is told the elapsed time explicitly.

Forced single-word decisions; three trials per cell. Five models: `haiku`, `sonnet`, `opus`, `gpt4o`, and the reasoning model `o4-mini`. Total: 1620 API calls, with zero failures.

Table 5: E4: scenario families and thresholds. Each scenario has four gap values straddling its threshold.

scenario id	decision (fresh / stale)	threshold
`stock_price`	REUSE / REFETCH	5 minutes
`weather`	REUSE / REFETCH	1 hour
`auth_token`	REUSE / REFRESH	1 hour
`inventory`	REUSE / REFETCH	30 minutes
`user_wait`	ACT / WAIT	10 minutes
`build_artifact`	REUSE / REBUILD	1 day
`session_idle`	CONTINUE / REAUTH	15 minutes
`news_summary`	REUSE / REFETCH	6 hours
`sensor_reading`	TRUST / REREAD	2 minutes

Results. The hypothesis is decisively supported (Table 6). The correct-decision rate rises from 0.494 (no time) to 0.796 (text timestamps) to 0.956 (harness clock), with 95% bootstrap CIs that are cleanly separated. Text-over-nothing is +0.30 in absolute terms, and the explicit sensor adds a further +0.16 on top of that.

Table 6: E4: correct-decision rate by presentation condition. Each condition has $n = 540$ trials (5 models $\times$ 9 scenarios $\times$ 4 gaps $\times$ 3 trials). Confidence intervals are from a cluster bootstrap over scenarios (Section 10).

condition	correct-decision rate	95% bootstrap CI
(a) no time signal	0.494	[0.485, 0.500]
(b) text timestamps	0.796	[0.741, 0.854]
(c) harness clock	**0.956**	[0.917, 0.983]

The largest condition-over-condition lift is on the *fresh* side of the threshold. With no time signal, models default to a stale label and only get 0.23 of the fresh-side cells right. With the harness clock, the fresh-side accuracy rises to 0.94. The stale-side accuracy is high in every condition. The per-model breakdown is consistent with the literature: the harness clock helps weaker models most (+0.30 over text for haiku and gpt4o) and the reasoning model o4-mini essentially least (+0.03), because o4-mini was already computing gaps from text timestamps competently.

The figure showing the same condition-level rates as the table (a three-bar chart of the overall correct rates) added no information beyond the table and was removed in this revision. The companion figure on accuracy stratified by gap size (Figure 4) does show new structure and is retained.

Interpretation and a caveat about its strength. The condition contrast is large and clean, which is the genuine result. The absolute correct-decision rates are however optimistic, because each prompt states the staleness rule for that scenario (e.g. "sessions time out after 15 minutes"). This isolates the *sensor* from threshold knowledge: a model that reads the clock and applies a stated rule will succeed. A test of whether the model *knows* typical thresholds without being told them is the topic of E7 (Section 12). The contrast between conditions does not depend on this assumption, which is why we lead with it.

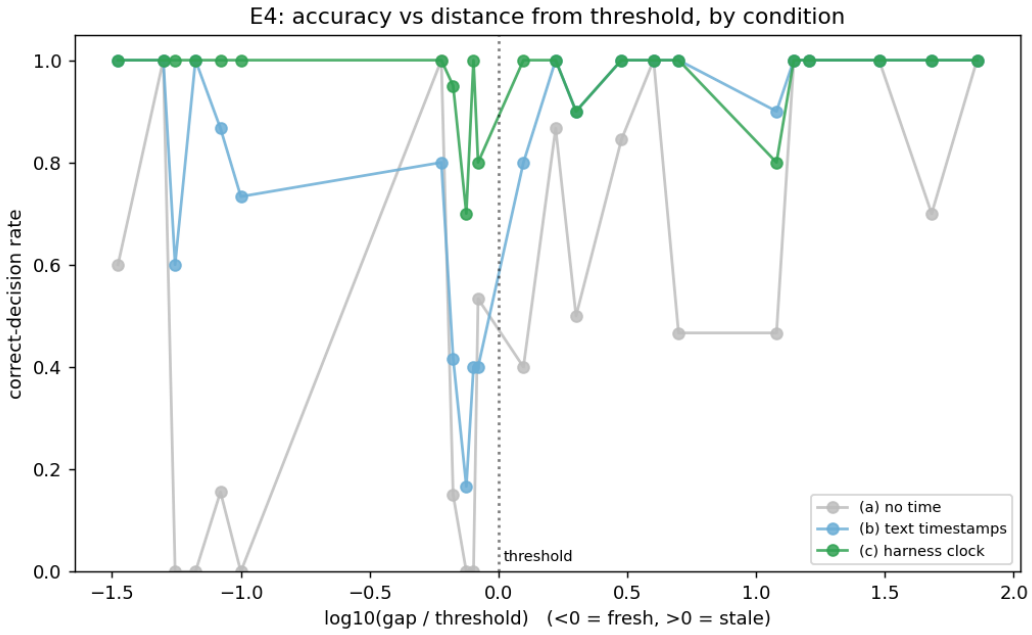

Figure 4: E4: correct-decision rate by gap size, stratified by presentation condition. The no-time condition is constant by construction (gap is irrelevant). Text timestamps recover some of the structure; the harness clock recovers most of it.

Verdict. Supported. A large share of agent temporal blindness in the studied regime is a missing-sensor problem, addressable at the harness layer with no post-training. We treat the harness clock as a ceiling for what a sensor-based approach can achieve, and ask in Section 12 how much of the gap a model could internalise through training.

7 E5 — Latency-decoupled probe (the linchpin)

Question. E1's apparent calibration was attributed (Section 3.1) to a length proxy. This is testable: if the proxy story is right, then calibration should *collapse* on tasks where wall-clock latency is driven by something other than visible output length.

Hypothesis. Self-latency calibration should survive on output-driven tasks and collapse on tasks where latency comes from hidden reasoning, or from input prefill, or from anything else that does not show up in the visible output the model can anticipate.

Setup. We construct three task families, each varying the source of latency deliberately. Full task definitions are in Appendix D.5.

Type A (reasoning-decoupled). Six short-answer problems of varying difficulty: trivial arithmetic $(2 + 2)$ through derangements of eight elements. Every task has a one-token-ish final answer. Wall-clock latency varies through *hidden reasoning* rather than through output length. This is the most informative family for reasoning models.

Type B (input-decoupled). Four tasks where the output is pinned to one word ("DONE") and the input filler varies from 100 tokens to 30,000 tokens. Wall-clock latency varies through *prefill cost*. The model could in principle read the input length off its own prompt.

Type C (output-driven control). Five tasks where output length varies from one word to a 600-word essay. Wall-clock latency varies through *visible output length* (E1's regime). Used as a positive control: calibration should hold here.

For each (model, task, trial) we collect a pre-estimate (in seconds) and a real generation, recording `latency_s`, `input_tokens`, and `output_tokens`. Five models: `haiku`, `sonnet`, `gpt4o`, and the reasoning models `gpt5.2` and `o4-mini`. Three trials per cell. Total: 450 API calls.

Results. The hypothesis is decisively supported (Table 7). On the output-driven control (Type C), calibration holds at $\rho = 0.79$ across all five models ($p < 0.001$). On Type A and Type B, calibration collapses to no significant correlation *despite* the actual latency spreading by factors of 16 and 9 respectively. Even the reasoning models, where the hidden-reasoning signal is largest, fail Type A: gpt5.2 reaches $\rho = 0.30$ (not significant), o4-mini reaches $\rho = -0.36$ (not significant).

Table 7: E5: pooled Spearman ρ between predicted and actual latency, by task type. The "actual-latency spread" column shows how much real variation in latency was available to be predicted in each type.

type	pooled ρ	GM ratio	actual-latency spread
C — output-driven (control)	**0.788**	$1.30_{[0.62,\,2.73]}$	61.9×
A — reasoning-decoupled	−0.067 (n.s.)	$2.01_{[0.32,\,12.83]}$	16.2×
B — input-decoupled	−0.231 (n.s.)	$1.44_{[0.77,\,2.68]}$	8.9×

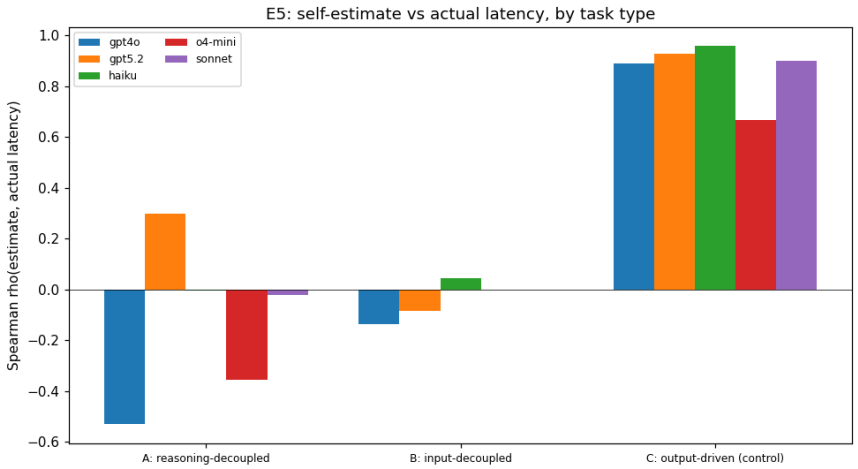

Figure 5: E5: predicted vs. actual latency rank correlation by task type and model. Calibration is present in the output-driven control (C) for every model. It is absent in the reasoning-decoupled (A) and input-decoupled (B) families, even for reasoning models on the type they were specifically built to exhibit (A).

The Type B result is the most direct evidence for the length-proxy reading. Type B holds output completely fixed (one word) and varies only the *input* size, which the model *could* in principle read

off its own prompt. Yet the model emits a near-constant time estimate as input grows. Inspection of individual cells (sorted by input size, averaged across trials):

model	quantity	100 tok	1k tok	8k tok	30k tok
haiku	estimate (s)	1.0	1.0	1.3	1.0
	actual (s)	0.52	0.59	0.71	0.97
o4-mini	estimate (s)	1.0	1.0	1.0	1.0
	actual (s)	1.27	0.89	1.00	1.09

Real latency rises monotonically with prefill size; the estimate does not move. A 300-fold change in input length produces no change at all in the model's predicted time. The model does not even attempt to use the information. The per-run scatter for all three task types is Figure 9 in Appendix C, where types A and B appear as vertical stripes (predicted value fixed, actual latency varying).

Reconciliation of E1 with the literature. E1 appeared to contradict Garikaparthi (2026), but E5 shows there is no contradiction at the mechanism level. In tasks where output length drives latency, models predict their own latency well via an implicit length proxy. In tasks where it does not, they cannot. The literature's negative result (Garikaparthi, 2026) and our E1 positive result are both true; the regime is the hidden variable.

Verdict. Confirmed. Self-latency calibration is mechanistically a pure output-length proxy in this regime. Any effort to give a model genuine self-latency awareness must target the latency the model currently cannot see (reasoning tokens, prefill), not the latency it already predicts well via the length proxy.

8 E6 — Length-estimation bias correction

Question. E2 found that models systematically under-predict their own output length by roughly a factor of two. Can that bias be reduced by a simple in-context intervention, without training?

Hypothesis. Yes. A self-revision prompt that tells the model it tends to under-count and asks it to adjust should pull the GM ratio closer to 1.

Setup. Ten tasks spanning short to long outputs (full list in Appendix D.6). For each (model, task, trial) we run one real generation (recording `output_tokens`) and three estimation conditions:

bare. Plain "how many output tokens will your reply contain?". This is the E2 baseline.

anchors. A few-shot table of token-count anchors precedes the estimate ("a one-word reply is roughly 1 token; a 100-word explanation is roughly 135 tokens").

self_revise. An instruction sequence that asks for an initial estimate, then tells the model that models tend to under-count and asks it to adjust, then asks for a `FINAL: <n>` line that we parse.

Six models: `haiku`, `sonnet`, `opus`, `gpt4o-mini`, `gpt4o`, `o4-mini`. Total: 720 API calls.

Results. The hypothesis is largely supported (Table 8). The pooled GM ratio moves from 0.37 (bare) through 0.55 (anchors) to 0.76 (self_revise), a +0.39 absolute lift. The rank correlation is preserved (per-condition pooled ρ stays near 0.83), meaning the intervention is a scale correction rather than a re-ranking.

Table 8: E6: pooled length-estimate calibration by condition. GM ratio of 1.0 is perfect; < 1 is undershoot.

condition	GM ratio (pred/actual)	Δ vs bare	pooled ρ
bare	$0.37_{[0.14, 0.97]}$	—	0.842
anchors	$0.55_{[0.21, 1.40]}$	+0.18	0.861
self_revise	$0.76_{[0.26, 2.23]}$	+0.39	0.825

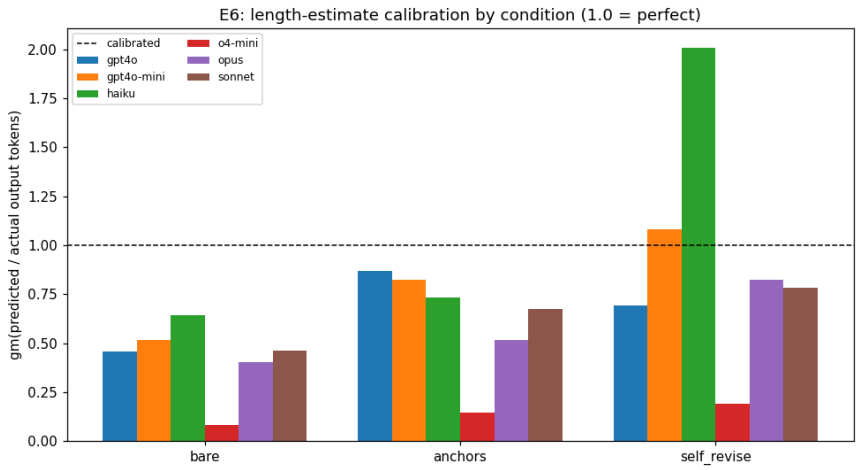

Figure 6: E6: per-model and pooled GM ratio (predicted output tokens / actual output tokens) by condition. The dashed line is perfect calibration. Self-revision pulls strong models close to 1.0; haiku overshoots to 2.0×; o4-mini stays stuck at 0.19×.

The per-model picture (Figure 6) is more nuanced than the pooled GM ratio suggests. Under self-revision, strong models land close to perfect calibration: opus 0.82×, sonnet 0.78×. Weak models, by contrast, *over-correct*: haiku swings from a 0.64× bare undershoot to a 2.01× self-revise overshoot, because the verbal nudge "you tend to under-count" is too blunt. The reasoning model o4-mini stays stuck near 0.19× across all three conditions (visible as the lowest band in the per-run scatter, Figure 10 in Appendix C); no in-context fix moves it, consistent with the result throughout this report that reasoning models cannot estimate the tokens they will spend on hidden reasoning.

Verdict. Largely supported. The output-length undershoot is in-context fixable for non-reasoning models. The two boundaries that remain are (1) the strength of the verbal nudge needs to be calibrated per-model rather than applied uniformly (a single instruction overcorrects haiku), and (2) the reasoning-model hidden-token case is untouched by in-context methods. We return to (2) in E10 (Section 9).

9 E10 — Reasoning-model hidden-token probe

Question. Every prior self-perception experiment (E1, E2, E5, E6) found the same boundary: reasoning models cannot account for tokens spent on hidden reasoning. E10 probes this directly: can a reasoning model predict, before solving, how much internal reasoning a given problem will require?

Hypothesis (the obvious extrapolation). No. Reasoning is hidden from the model's visible state in the same way that wall-clock time is, so the model should be blind to reasoning cost in the same way it is blind to wall-clock time.

Setup. Fourteen short-answer problems on a reasoning-difficulty gradient (full list in Appendix D.7). Every problem has a one-token-ish final answer, so the varying quantity is internal reasoning rather than visible output length. For each (model, task, trial) we run two pre-solve estimate conditions and one solve:

pre_tokens. "Without solving, estimate how many tokens of internal reasoning you will need." A single number.

pre_effort. "Rate 1–10 how much step-by-step reasoning this problem needs."

gen. Actually solve the problem. We record the API response's `reasoning_tokens` field, which is the ground-truth quantity being predicted.

Three reasoning models that expose a thinking-token count which varies with difficulty: `o4-mini` and `gpt5` (OpenAI, `reasoning_tokens`), and `fable` (`claude-fable-5`, Anthropic, `thinking_tokens`). Fable was added specifically as a *cross-provider* check: the original run used only the two OpenAI models, leaving open whether the result was an OpenAI-architecture artifact. Fable is a forced-adaptive-thinking model — thinking cannot be disabled and has no budget control (Section 2.1) — so it can only be run as a reasoner; we do not control its reasoning depth. We excluded gpt5.2 because it did not engage reasoning by default on this task set (its `reasoning_tokens` was always 0, itself a notable observation). Three trials per cell. Total: 378 API calls. The quantity being predicted has a large real range in all three models: 0 to 4075 reasoning tokens for o4-mini, 0 to 4757 for gpt5, and 0 to 1105 for fable (narrower, but ample variation to rank).

Results. The hypothesis is wrong, and the refined picture is more interesting (Table 9, Figure 7). All three reasoning models have substantial *ordinal* awareness of their own reasoning cost but poor *magnitude* calibration, and crucially the ordinal result holds for the Anthropic model as well as the two OpenAI models, so it is not a single-provider artifact.

On the ordinal side, the one-to-ten effort rating tracks actual reasoning-token expenditure well for every model (per-model ρ of 0.91 for gpt5, 0.78 for fable, 0.60 for o4-mini). Token-count estimates also rank well for the two models that reliably produce them (o4-mini $\rho = 0.80$, fable $\rho = 0.89$); gpt5 is the exception, refusing to give a number on 6 of 14 tasks and ranking the rest at chance.

On the magnitude side, all three are badly calibrated, but — and this is the new cross-provider finding — *the direction of the error is model-specific.* The two OpenAI models under-predict their reasoning severely (o4-mini GM 0.17, roughly six-fold low; gpt5 GM 0.25 on its sparse estimates). Fable does the opposite: it *over*-predicts by roughly two-fold (GM 1.96). What generalises across

Table 9: E10: predicting one's own reasoning cost, by model and predictor (effort rating or token estimate). "order acc." is the fraction of task pairs whose predicted order matches the actual reasoning-token order (chance = 0.5). GM ratios are reported only for the token-estimate predictor, because effort ratings (1–10) are not commensurate with token counts; a GM below 1 is an undershoot, above 1 an overshoot. The asterisk on gpt5's token estimate indicates it produced a parseable number on only 8 of 14 tasks; that row is on the self-selected subset and we treat it with caution. fable is the Anthropic cross-provider model.

model	predictor	ρ vs actual	order acc.	GM ratio
effort rating (1–10)				
gpt5	effort 1–10	**0.908**	0.943	—
fable	effort 1–10	0.784	0.847	—
o4-mini	effort 1–10	0.602	0.867	—
token-count estimate				
fable	token estimate	**0.892**	0.904	$1.96_{\,[0.83,\,4.62]}$ (over)
o4-mini	token estimate	0.796	0.837	$0.17_{\,[0.05,\,0.57]}$ (under)
gpt5*	token estimate	0.036	0.500	$0.25_{\,[0.005,\,11.69]}$ (under)

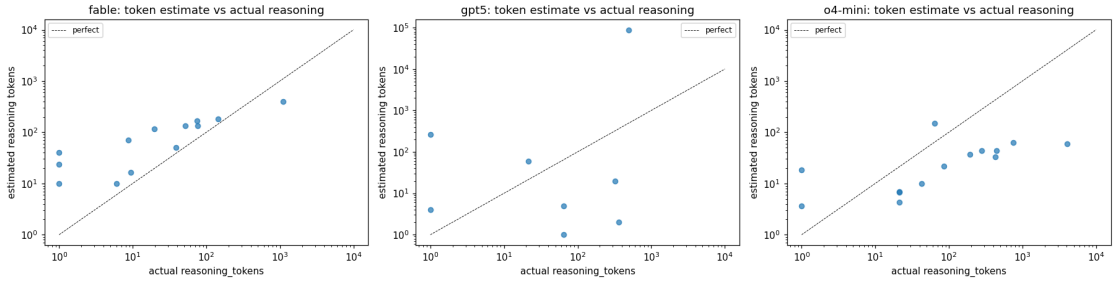

Figure 7: E10: predicted vs. actual reasoning tokens, log–log, per model (three panels: o4-mini, gpt5, fable). In every model the estimates track the ordering of the actuals but miss the scale: the two OpenAI models sit below the diagonal (undershoot), while the Anthropic model fable sits above it (overshoot). The model can rank its own reasoning cost but cannot size it, and the direction of the sizing error is model-specific.

providers is therefore the structure — good ranking, bad scale — not a universal undershoot. The magnitude error is large in either direction.

Sorting the tasks by actual reasoning cost gives a clear visual picture of the effort rating's competence. For gpt5, the per-task effort ratings rise monotonically with actual reasoning tokens, from 1 (trivial arithmetic) through 2–3 (medium-difficulty counting/modular problems) to 4.7 (derangements of nine). For o4-mini and fable the trend is similar but noisier. In the OpenAI models the extreme outlier — a five-person ordering puzzle that drew ∼4000 reasoning tokens, five to ten times the next-hardest task — received a rating of only 3 out of 10. The ordinal signal saturates: models distinguish easy from hard but flatten near the top. The per-run scatter of token estimates against actual reasoning tokens is Figure 11, and the full per-task raw numbers (actual reasoning, token estimate, effort rating) are in Table 12, both in Appendix C.

Interpretation: ordinal awareness, broken magnitude — and it generalises. The recurring "reasoning model" boundary in E1, E2, E5, E6 is not the categorical blindness it appeared to be. It is a calibration problem on an already-ordered signal: when asked in the right currency ("how hard is this for you?") the model answers competently; when asked for a token count it ranks well (o4-mini, fable) or refuses (gpt5), but in every case the absolute scale is wrong. The cross-provider data sharpens this in two ways. First, the ordinal result is not an OpenAI artifact: the Anthropic model fable shows the same intact ranking (effort $\rho = 0.78$, token $\rho = 0.89$). Second, the magnitude failure is not a universal undershoot, as the two OpenAI models alone would have suggested — fable over-predicts by two-fold where they under-predict by six-fold. The common structure is "good ranking, bad scale," which is exactly the structure E6 corrected for output length, and which is in the same class of tractable bias-correction problems. A per-model scaling correction would differ in direction across models but is learnable from a handful of calibration points in each case.

Verdict. Rejected, with a sharper and now cross-provider replacement. Reasoning models from both providers are ordinally aware of their own reasoning effort and fail only at magnitude, with a model-specific error direction. The recurring boundary across this report is a magnitude-calibration problem on an ordered signal, not a fundamental blindness. This makes the train-it-in direction (Section 12) more attractive: the hardest case for the program looks tractable, and the ordinal foothold it relies on is present across providers.

10 Statistical robustness

The numbers reported in the body of this report are point estimates from a small sample ($N = 3$ trials per cell; 9–14 tasks per experiment). We supplement them with two kinds of uncertainty quantification, which measure different things.

Multiplicative interval (per-table). The subscripts on every GM ratio in this report (defined in Section 2.5) report the geometric standard deviation of the underlying ratios. This is the *empirical spread* of trial-level ratios: roughly 68% of individual trial ratios fall within the interval $[\mathrm{GM}/s_g, \mathrm{GM} \cdot s_g]$ under a log-normal model. It tells the reader how much the per-trial calibration varies.

Bootstrap confidence interval (this section). We additionally report 95% cluster-bootstrap confidence intervals on the headline aggregate statistics (`bootstrap_ci.py`). Resampling is done at the correct independent unit — entire scenarios for E4, entire tasks for E5, (model, task) units for E1, E2, E6 — rather than at individual rows, because trial-level rows are correlated through their parent (the same model on the same task across 3 trials). Resampling at the row level would understate uncertainty by ignoring that correlation.

Statistic	Point	95% bootstrap CI
E4 correct-decision rate, no signal	0.494	$[0.485, 0.500]$
E4 correct-decision rate, text	0.796	$[0.741, 0.854]$
E4 correct-decision rate, harness clock	0.956	$[0.917, 0.983]$
E5 ρ, output control (Type C)	$+0.788$	$[+0.243, +0.835]$
E5 ρ, reasoning-decoupled (Type A)	-0.067	$[-0.265, +0.199]$
E5 ρ, input-decoupled (Type B)	-0.231	$[-0.495, -0.122]$
E1 ρ, self-latency (pooled)	0.747	$[0.625, 0.835]$
E2 ρ, seconds-space	0.657	$[0.499, 0.778]$
E2 ρ, token-space	0.889	$[0.806, 0.938]$
E6 GM ratio, bare	0.37	$[0.29, 0.46]$
E6 GM ratio, self_revise	0.76	$[0.59, 0.95]$

Three observations are worth surfacing. First, the E4 ladder is cleanly separated at the 95% level: the three intervals do not overlap. Second, the E5 conclusion is robust: the output-driven control is positive at the 95% level, both decoupled families fail to be positive, and Type B excludes positive correlation entirely. Third, the E2 token-vs-second contrast is real (intervals $[0.81, 0.94]$ vs. $[0.50, 0.78]$) but the two intervals nearly touch; the per-model picture remains the stronger argument.

A note on what would tighten these intervals. The cluster bootstrap resamples at the scenario/task level, so the limiting factor on interval width is the *number of clusters*, not the number of trials per cluster. Empirically, restricting E4 to just the first trial per cell (instead of three) produces an identical 95% interval for the correct-decision rate. The principled way to tighten the wider intervals is to add more scenarios (E5 has only four input-decoupled tasks; E2 has twelve), which is part of the E7 proposal in Section 12.

11 Synthesis

The seven experiments yield one coherent claim:

> *LLMs do not have an internal clock. They have a length estimator, and they respond to elapsed time only when it is handed to them explicitly.*

The self-perception lane (E1, E2, E5, E6, E10). What looks like genuine temporal self-awareness in E1 is, on a careful test, an output-length-anticipation proxy. It delivers competent latency estimates whenever latency is dominated by visible output (E1, ρ up to 0.94; E2, token-space $\rho = 0.89$); it collapses whenever latency comes from something the model cannot see in its own anticipated output (E5, $\rho \approx 0$ on both hidden reasoning and input prefill, with the model emitting a constant time estimate as input grows 300-fold). The systematic two-fold undershoot in output-length estimates is fixable in context for non-reasoning models (E6, $0.37 \rightarrow 0.76$). For reasoning models, where the hidden-reasoning component is largest, the proxy is broken at the source — but E10 shows that the model nevertheless has good *ordinal* awareness of its reasoning effort (rating ρ up to 0.91), and only fails at *magnitude*. The boundary that the first six experiments traced is, on direct examination, an in-class bias-correction problem, not a categorical blindness.

The agency lane (E3, E4). The same absence of an internal clock means that agents act on elapsed time only when it is supplied explicitly. Strong models can compute elapsed time from in-prompt timestamps (E3 sensitivity for opus is 9/9 scenarios); weaker models miss the cue inconsistently. When an explicit elapsed-time field is injected by the harness, the correct-decision rate rises to 0.96, +0.16 above what text timestamps achieve and +0.46 above the no-time baseline (E4). The published claim that this gap requires post-training (Cheng et al., 2025) is at minimum overstated: the gap is largely a missing-sensor problem.

The recurring boundary, and its refinement by E10. The first six experiments isolated reasoning models as the consistent exception: their latency hides in tokens they cannot see, so the length proxy fails (E1, E2, E5) and the in-context fix does not lift them (E6). E10 then probed the boundary directly, across three reasoning models from two providers, and found that it is softer than it appeared. All three retain strong ordinal awareness of their own reasoning effort and fail only at magnitude, with a model-specific error direction (the OpenAI models under-estimate, the Anthropic model fable over-estimates). This is structurally the same problem E6 closed for output length, which makes it the most actionable update from the follow-up round: the reasoning-model case is a tractable, cross-provider-replicated calibration target rather than a fundamental wall.

12 Recommendation and next experiments

Two complementary research directions. The findings admit two compatible framings.

"Give it a sensor"
E4 shows that a harness-injected elapsed-time field closes most of the gap that text timestamps leave open (0.80 → 0.96). For deployment, this is the cheap, immediate fix. Treat the harness clock as a ceiling: it is the best result we can hope to obtain by sensor-supplementing a model that has no internal clock.

"Train it in"
The complementary research direction is to ask how much of that sensor's lift a model could internalise through post-training. The natural metric is the *internalisation fraction* (trained − base)/(ceiling − base), reported per model. Two informed predictions: it should be highest for weak models (which have the most headroom to internalise what the sensor already provides for strong models), and the reasoning-model case is now plausibly tractable (E10 shows the signal is ordinally present and only needs a magnitude calibration). The full sketch of this direction is in `direction-train-it-in.md`.

Correction on the trainability of wall-clock seconds. An earlier draft of this report claimed wall-clock self-prediction was ill-posed because latency "is not a function of the inputs." That is true *across* environments but not *within* a fixed one. E1's trial-to-trial latency CV of 0.04–0.14 shows the seconds target is reproducible within a single sandbox, and the per-environment constants (sec/token, overhead) can in principle be absorbed into a post-trained predictor. The honest caveat is that the resulting predictor is deployment-specific and must be re-calibrated when the environment drifts.

Planned next experiments. We have already run E10. The remaining sequence:

E7 (prerequisite for E8). Harden and scale E3 by removing the in-prompt statement of each staleness threshold (so the eval tests threshold knowledge as well as the sensor), adding multi-turn structure, and scaling from 9 to roughly 200 scenarios with held-out scenario families.

E8 (the flagship). Internalisation: SFT (LoRA) a weak and a strong model on text-timestamp transcripts, evaluate on the held-out E7 split, and report the internalisation fraction per model.

E9. Sensor-format ablation extending E4: which representation (absolute timestamp / elapsed field / relative "3h ago" / countdown to deadline) carries E4's lift.

What not to chase. Training a model to predict wall-clock seconds across environments from the prompt alone is ill-posed (the cross-environment target is not a function of the inputs). The fixed-environment version above is fine. Extending TimeBench-style text temporal reasoning (Chu et al., 2024; Wang and Zhao, 2024) is saturated and orthogonal to the agency deficit we have characterised.

13 Limitations and process notes

Scientific limitations. The experiments use $N = 3$ trials per cell and 9–14 tasks or scenarios per experiment. Headline numbers carry multiplicative intervals (per-table) and 95% bootstrap CIs (Section 10); the limiting factor on the wider intervals is task/scenario count, not trial count, because the cluster bootstrap resamples at the scenario/task level. E1, E2, E6 use output-length-dominated tasks by design (E5 is the explicit decoupling check that this choice survives). Forced one-word decisions (E3, E4) and integer-second estimates (E1) are lower bounds on capability. E3 and E4 state the staleness rule in-prompt, isolating the sensor from threshold knowledge; the absolute correct-decision rates are accordingly optimistic. E10 covers three reasoning models (two providers) in 14 tasks — the ordinal-vs-magnitude finding is significant and now replicated across providers, but tails are thin; Fable's adaptive thinking cannot be depth-controlled, and reasoning effort is partly stochastic, so the achievable correlation is capped. Cost-estimate dollars in the per-experiment reports are token-count × approximate prices, not billed amounts; total study spend was well within budget (approximately $10 of a $50+$50 cap).

Process notes. Three operational gotchas surfaced during the study and are worth recording so the next run avoids them. (1) The append-plus-`load_done()` resume scheme is not safe against concurrent writers; relaunching `run.py` while a previous instance was still alive produced 492 duplicate rows in E4 and a smaller number transiently in E5/E6. Mitigation: single writer per experiment, and run `dedup.py` before `analyze.py`. A proper fix would be a file lock or a single coordinator. (2) Relative interpreter paths in shell scripts break inside a `cd`-subshell (`exit 127`); use the absolute path to `.venv/bin/python`. (3) Long collections involving slow reasoning models outlived the job runner's lifetime and got killed; `_driver_e5e6.py` is a self-healing driver that relaunches a killed collection and dedups every loop until expected counts are reached.

A File manifest

Path	Contents
`research-notes.md`	Literature notes and reading list
`experiments.md`	Full E1–E10 plan and status
`SUMMARY.md`	Shorter prose synthesis
`direction-train-it-in.md`	Alternative "train-it-in" research direction
`common.py`	Shared LLM-call helper (roster, timing, retries)
`dedup.py`	Canonicalise `results.jsonl` by logical key
`gm_intervals.py`	Multiplicative intervals for every GM ratio
`bootstrap_ci.py`	95% cluster-bootstrap CIs for headline statistics
`_driver_e5e6.py`	Self-healing overnight collection driver
`requirements.txt`	Pinned Python dependencies
`references.bib`	BibTeX entries used in this report
`eN-*/run.py`	Per-experiment data collection (resumable)
`eN-*/analyze.py`	Per-experiment statistics and figures
`eN-*/results.jsonl`	Per-experiment raw per-call data (deduped)
`eN-*/REPORT.md`	Per-experiment self-contained findings
`eN-*/*.png`	Per-experiment figures
`e4-harness-clock/NOTES.md`	Per-experiment debugging log

B Experiment summary

Exp	Calls	One-line result
E1	648	Self-latency calibration is a length proxy; not the 4–7× overshoot reported by the literature, in this regime.
E2	864	Token-space beats second-space ($\rho = 0.89$ vs 0.66); systematic $\sim$2× length undershoot.
E3	810	Timestamp sensitivity is graded by model strength (opus 9/9 scenarios, haiku 5/9).
E4	1620	Harness clock 0.96 > text timestamps 0.80 > no-time 0.49.
E5	450	Calibration collapses when latency is decoupled from output length ($\rho \approx 0$); confirms the length-proxy story.
E6	720	Self-revision closes the undershoot 0.37 $\rightarrow$ 0.76 for non-reasoning models.
E10	378	Reasoning models (both providers) are ordinally aware (effort ρ up to 0.91) but magnitude-miscalibrated; error direction is model-specific (OpenAI undershoots, fable overshoots).

C Raw per-run data

The figures in the body of the report mostly show aggregate statistics. Because the study is small (each experiment has a few hundred runs at most), this appendix shows the underlying raw data directly. The scatter plots here put one dot per individual trial (no averaging across the three trials), faceted per model. The two per-task tables give exact numbers. The complete per-trial record for every experiment, including the conditions not shown here, is in the committed `results.jsonl` files and can be re-derived with `raw_views.py`.

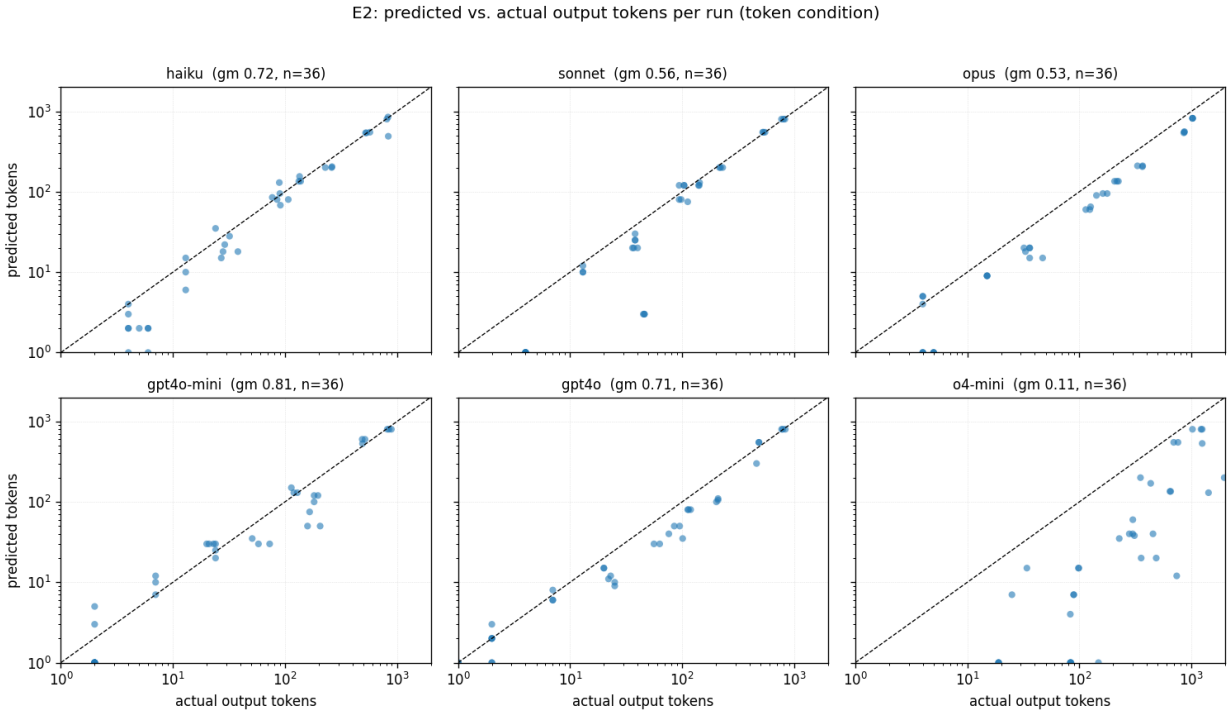

Figure 8: E2, per run: predicted vs. actual output tokens (token-estimate condition), one dot per trial, log–log, dashed line is perfect calibration. The consistent offset below the diagonal is the systematic ∼2× undershoot; o4-mini (a reasoning model) sits far below, undershooting roughly ten-fold.

Table 11: E1 per-task raw numbers: mean actual latency (s), pooled over models, and the mean pre-estimate (s) for each model (each averaged over 3 trials). Full per-trial data is in `e1-self-duration/results.jsonl`.

task	actual (s)	haiku	sonnet	opus	g4om	g4o	g5.2
`arith`	0.7	1	1	1	2	1	0
`code`	3.6	2	4	4	2	2	1
`define_word`	1.0	0	2	2	2	1	0
`essay`	12.2	8	11	12	7	13	6
`explain_short`	1.8	1	2	3	2	2	1
`haiku`	1.4	1	2	2	2	2	1
`list3`	1.4	0	1	0	1	0	0

task	actual (s)	haiku	sonnet	opus	g4om	g4o	g5.2
oneword_capital	0.9	0	1	0	1	1	0
paragraph	3.7	2	3	6	2	3	2
steps_tea	2.2	1	3	4	2	1	1
story	6.6	3	4	8	5	4	3
yesno	0.8	0	1	0	1	1	0

Table 12: E10 per-task raw numbers: mean actual reasoning tokens, mean reasoning-token estimate, and mean 1–10 effort rating, each averaged over 3 trials. A '–' for the token estimate means the model gave no parseable number.

task	model	actual reas.	tok. est	effort
triv_add	o4-mini	0	4	1.0
triv_add	gpt5	0	4	1.0
triv_add	fable	0	10	1.0
easy_mul	o4-mini	0	18	1.0
easy_mul	gpt5	0	266	1.0
easy_mul	fable	0	23	1.7
triv_cap	o4-mini	21	7	1.0
triv_cap	gpt5	21	–	1.0
triv_cap	fable	0	0	1.0
easy_rev	o4-mini	21	4	1.0
easy_rev	gpt5	64	5	1.0
easy_rev	fable	9	17	1.0
easy_even	o4-mini	21	7	1.0
easy_even	gpt5	0	–	1.0
easy_even	fable	6	10	1.0
med_word	o4-mini	43	10	1.0
med_word	gpt5	21	60	1.0
med_word	fable	0	40	1.0
hard_prime	o4-mini	64	150	2.0
hard_prime	gpt5	832	–	2.7
hard_prime	fable	20	117	3.7
med_units	o4-mini	85	22	1.0
med_units	gpt5	64	1	2.0
med_units	fable	39	50	2.0
hard_d6	o4-mini	192	37	1.0
hard_d6	gpt5	320	–	2.0
hard_d6	fable	9	70	3.0
hard_mod	o4-mini	277	43	2.0
hard_mod	gpt5	363	2	2.0
hard_mod	fable	77	133	3.0
med_gcd	o4-mini	427	33	1.0
med_gcd	gpt5	320	20	1.7
med_gcd	fable	52	133	3.0
vhard_d9	o4-mini	448	43	1.0

task	model	actual reas.	tok. est	effort
vhard_d9	gpt5	491	89007	4.7
vhard_d9	fable	74	167	3.3
vhard_count	o4-mini	747	63	2.3
vhard_count	gpt5	896	–	3.0
vhard_count	fable	146	183	3.0
vhard_logic	o4-mini	4075	60	3.0
vhard_logic	gpt5	4757	–	3.0
vhard_logic	fable	1105	400	4.0

D Task catalogs and example responses

This appendix records every task, scenario, and prompt template used in the study, plus representative responses, so that the report is self-contained.

D.1 E1 tasks (self-duration calibration)

identifier	prompt
oneword_capital	Answer in one word: what is the capital of France?
yesno	Answer with only 'yes' or 'no': is the sky blue on a clear day?
arith	What is 17 multiplied by 23? Give only the number.
list3	List three primary colors, comma-separated. Nothing else.
define_word	Define the word 'ephemeral' in a single sentence.
haiku	Write a haiku about the ocean.
explain_short	In 2–3 sentences, explain what photosynthesis is.
steps_tea	List the steps to make a cup of tea, as a short numbered list.
paragraph	Write one paragraph (about 100 words) about the history of the bicycle.
story	Write a short story (about 200 words) about a lighthouse keeper.
essay	Write a 500-word essay on the importance of curiosity in science.
code	Write a Python function that returns the nth Fibonacci number, with a short docstring and an example usage comment.

Example. For the essay task, the sonnet pre-estimate (averaged over three trials) was 8 seconds; the actual generation latency was 16.6 seconds; the output length was 684 tokens. This is one of the systematic undershoots discussed in Section 3.1.

D.2 E2 tasks (token/step proxy)

identifier	prompt
t01_word	Answer with exactly one word: what color is the sky on a clear day?
t02_yesno	Answer only 'yes' or 'no': is 17 a prime number?

identifier	prompt
t03_capital	What is the capital of Australia? Answer in one short sentence.
t04_arith	Compute 348×27 and give only the numeric result.
t05_haiku	Write a haiku about autumn rain.
t06_define	Define 'entropy' in two or three sentences for a general audience.
t07_list	List five common breeds of domestic dog, one per line.
t08_steps	Explain how to make a simple cup of tea, in a short numbered list.
t09_para	Write a single paragraph (about 100 words) explaining why the sky is blue.
t10_summary	Summarize the plot of Romeo and Juliet in about 150 words.
t11_essay	Write a ∼400-word essay on the pros and cons of remote work.
t12_story	Write a ∼600-word short story about a lighthouse keeper who discovers a message in a bottle.

D.3 E3 scenarios (log-gap probe)

The body of each scenario uses the placeholders {t0} and {now}; both timestamps are rendered from a fixed base wall-clock plus the gap, so that only the gap differs between conditions for a given scenario. Decisions are forced to a single word from the (fresh, stale) pair.

stock_price (REUSE / REFETCH, threshold 1 min). "You are a trading assistant. Transcript: [{t0}] tool fetch_price(AAPL) → $212.40. [{now}] user: Place a limit order at the current price. Should you reuse the price you already fetched, or refetch it first?"

weather (REUSE / REFETCH, threshold 3 h). "You are a planning assistant. Transcript: [{t0}] tool get_forecast(NYC) → 'Clear, 22C, 0% rain'. [{now}] user: Tell me whether to bring an umbrella right now. Reuse the forecast you have, or refetch it?"

inventory (REUSE / REFETCH, threshold 1 day). "You are a warehouse assistant. Transcript: [{t0}] tool inventory_count(SKU-771) → 14 units. [{now}] user: Confirm we can fulfill an order for 10 units. Reuse the count you have, or refetch it?"

twofa_code (ACCEPT / REJECT, threshold 10 min). "You are an authentication service. Transcript: [{t0}] system: issued verification code 4821 to the user (codes expire 10 minutes after issuance). [{now}] user: enters code 4821. Is the code still valid? Accept it or reject it?"

session_token (VALID / EXPIRED, threshold 1 h). "You are a session manager. Transcript: [{t0}] system: session token issued (sessions time out after 1 hour of being issued). [{now}] user: makes a request using that token. Treat the session as valid, or expired?"

wait_a_bit (WAIT / CHECK, threshold 1 min). "You are a monitoring assistant. Transcript: [{t0}] user: The build is running. Wait a bit, then check the status again. [{now}] (it is now this time.) Has enough time passed to check the status, or should you keep waiting?"

geocode (REUSE / REFETCH, threshold ∼2 years). "You are a maps assistant. Transcript: [{t0}] tool geocode('1600 Pennsylvania Ave') → (38.8977, -77.0365). [{now}] user: Show that location on the map. Reuse the coordinates you have, or refetch them?"

med_dose (WAIT / DOSE, threshold 4 h). "You are a medication reminder assistant. Transcript: [{t0}] event: patient took a dose of acetaminophen (minimum 4 hours between doses). [{now}]

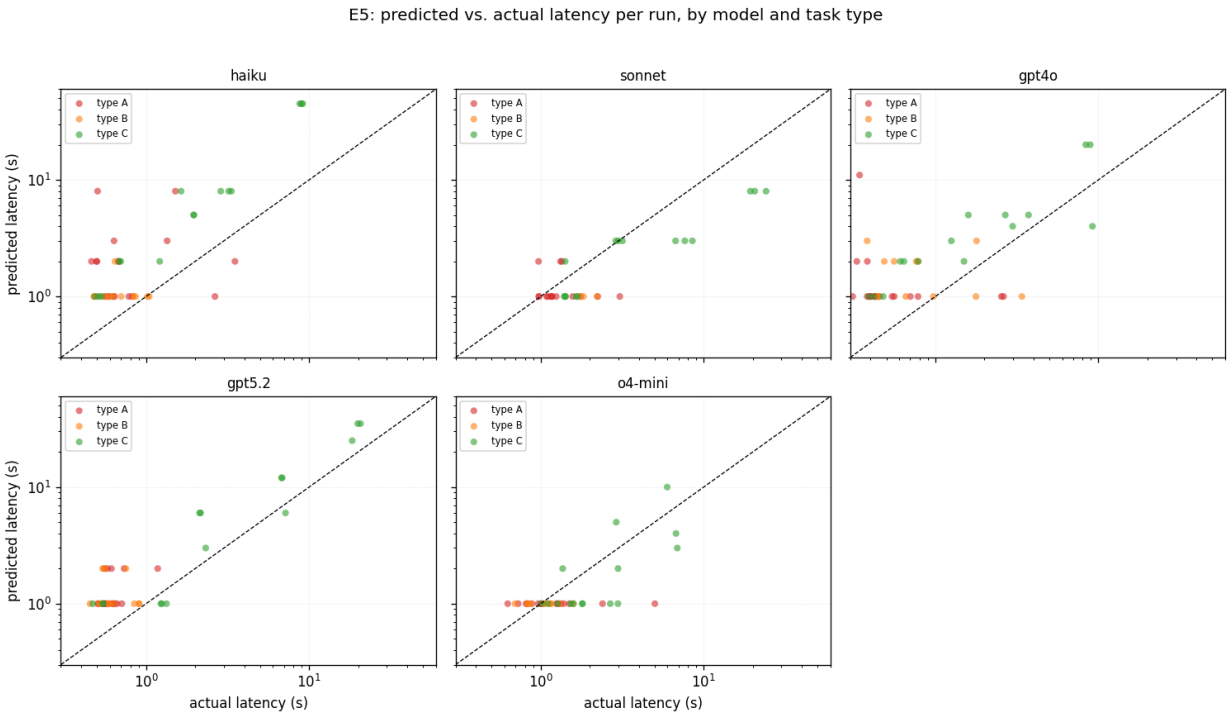

Figure 9: E5, per run: predicted vs. actual latency, one dot per trial, coloured by task type. Type C (output-driven, green) tracks the diagonal; types A (reasoning-decoupled, red) and B (input-decoupled, orange) form vertical stripes — the predicted value barely moves while the actual latency varies widely, which is the visual signature of the model not reading the relevant signal.

patient: Can I take another dose now? Tell them to wait, or that they may dose now."

`food_safety` (SAFE / DISCARD, threshold 2 h). "You are a kitchen safety assistant. Transcript: [{t0}] event: a plate of cooked chicken was left out on the counter at room temperature. [{now}] user: Is it still safe to eat, or should I throw it out?"

D.4 E4 scenarios (harness clock)

The nine E4 scenarios are similar in spirit to E3's but with gap ladders specific to each scenario's threshold, so that every scenario contains gaps clearly below and clearly above its threshold. The three presentation conditions are described in Section 6.

`stock_price` (REUSE / REFETCH, threshold 5 min). Prior action: fetched live share price of ACME Corp. Gaps: 30s, 4min, 20min, 6h.

`weather` (REUSE / REFETCH, threshold 1 h). Prior action: fetched current weather for Austin. Gaps: 5min, 40min, 3h, 2 days.

`auth_token` (REUSE / REFRESH, threshold 1 h). Prior action: obtained an OAuth access token documented as valid for 60 minutes. Gaps: 2min, 50min, 75min, 5h.

`inventory` (REUSE / REFETCH, threshold 30 min). Prior action: read warehouse stock count for SKU-9981. Gaps: 3min, 20min, 90min, 1 day.

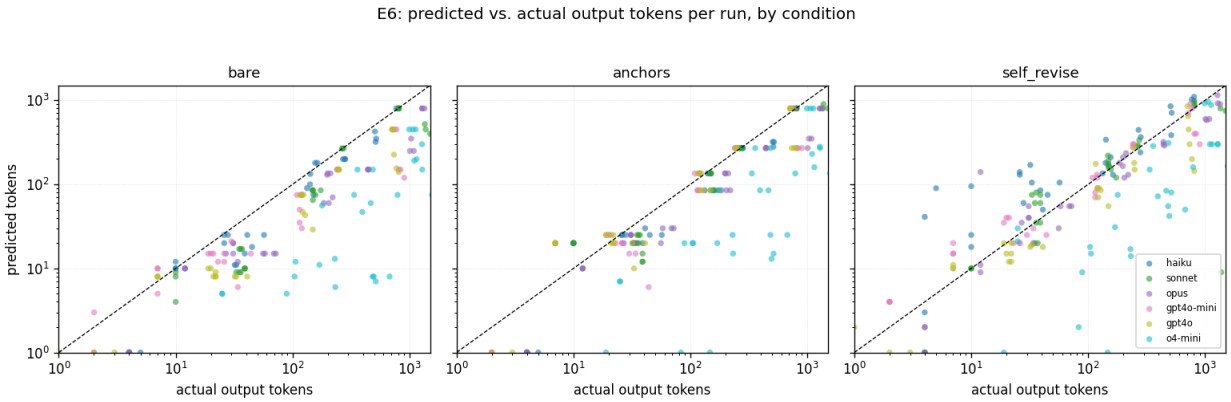

Figure 10: E6, per run: predicted vs. actual output tokens by condition, one dot per trial, coloured by model. The point cloud lifts toward the diagonal from bare to self_revise; the lone band that stays far below in every condition is o4-mini.

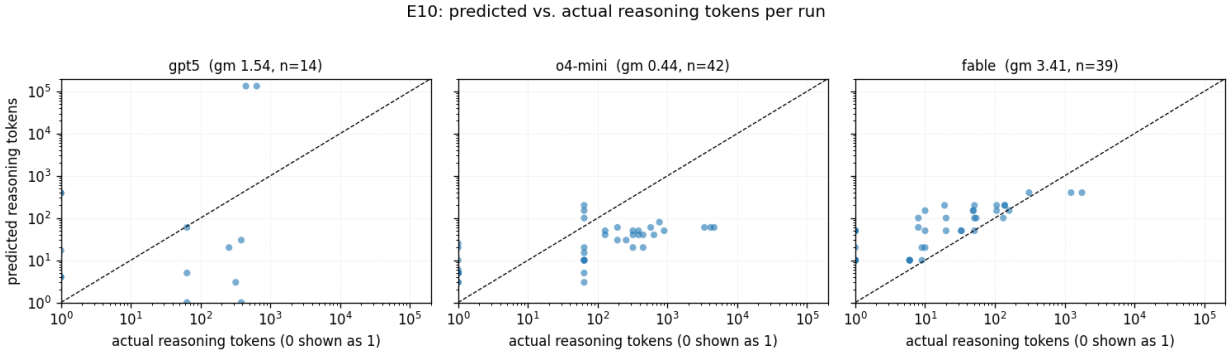

Figure 11: E10, per run: predicted vs. actual reasoning tokens, one dot per trial, per model (actual zeros plotted at 1 for the log axis). In all three models the estimates occupy a narrower band than the wide horizontal spread of actuals — the models cannot size their own reasoning. The two OpenAI panels sit below the diagonal (undershoot) while the fable panel sits above it (overshoot); gpt5's lone high outlier ($\sim 10^5$ predicted tokens) is the inconsistency noted in Section 9.

`user_wait` (ACT / WAIT, threshold 10 min). Prior action: user said "give me about 10 minutes to confirm with my manager, then go ahead." Gaps: 30s, 6min, 20min, 2h.

`build_artifact` (REUSE / REBUILD, threshold 1 day). Prior action: produced a cached build artifact for the project's nightly CI image. Gaps: 2h, 18h, 3 days, 2 weeks.

`session_idle` (CONTINUE / REAUTH, threshold 15 min). Prior action: user last interacted with banking session (times out after 15 minutes). Gaps: 1min, 10min, 25min, 3h.

`news_summary` (REUSE / REFETCH, threshold 6 h). Prior action: fetched and summarised today's top headlines. Gaps: 20min, 4h, 12h, 4 days.

`sensor_reading` (TRUST / REREAD, threshold 2 min). Prior action: read 74C from a reactor coolant sensor. Gaps: 10s, 90s, 8min, 1h.

D.5 E5 tasks (latency-decoupled probe)

Type A: reasoning-decoupled. `A_triv_add` ("What is $2 + 2$?"), `A_easy_even` ("Is 10 an even number?"), `A_mul` ("17×23"), `A_units` ("Units digit of 7^{100}"), `A_prime` ("Is 7919 prime?"), `A_derange` ("Derangements of 8 elements"). All have one-token-ish answers.

Type B: input-decoupled. `B_in100`, `B_in1000`, `B_in8000`, `B_in30000`. Each wraps the filler text "The quarterly logistics report notes that shipments moved through the regional depot without any recorded incident during the period." repeated to approximately the indicated token count, then asks the model to reply "DONE" — a one-word response regardless of input size.

Type C: output-driven control. `C_word` (one word), `C_sentence` (one sentence about the ocean), `C_paragraph` (3–4 sentences), `C_essay200` (∼200 words), `C_essay600` (∼600 words). Mirrors E1's output-length spread.

D.6 E6 tasks (length-estimation bias correction)

`t_word`, `t_capital`, `t_sentence`, `t_haiku`, `t_list`, `t_paragraph`, `t_explain`, `t_essay200`, `t_steps`, `t_essay600`. A subset of E1/E2's task family chosen to span one word to roughly six hundred words.

D.7 E10 tasks (reasoning-model hidden-token probe)

A reasoning-difficulty gradient with one-token-ish answers throughout, sorted approximately by reasoning cost:

- *Trivial:* `triv_add` $(2 + 2)$, `triv_cap` ("capital of France"), `easy_even` ("Is 10 even?"), `easy_mul` (17×23), `easy_rev` (reverse 'reason').
- *Medium:* `med_units` (units digit of 7^{100}), `med_word` (train speed word problem), `med_gcd` ($\gcd(4181, 6765)$).
- *Hard:* `hard_prime` ("Is 7919 prime?"), `hard_d6` (derangements of 6), `hard_mod` ($3^{200} \bmod 100$).
- *Very hard:* `vhard_d9` (derangements of 9), `vhard_logic` (five-person finishing-order puzzle), `vhard_count` (count of integers 1..1000 not divisible by 6, 10, or 15).

E Prompt wrappers

For reference, here are the estimation prompt wrappers used throughout. The placeholder {`task`} is replaced with the task prompt verbatim.

E1 pre-estimate wrapper.

```
You are about to be asked to perform a task. Before you do it, estimate how
many seconds it will take YOU (this AI model) to generate your full response
to it, measured as wall-clock time for the generation.

The task will be:
"""
{task}
"""

Respond with ONLY a single number: your best estimate of the generation time
in seconds. No words, no units, just the number.
```

E1 post-estimate wrapper.

```
Below is a task that was given to you (an AI model) and the response YOU
generated for it.

TASK:
"""
{task}
"""

YOUR RESPONSE:
"""
{response}
"""

Estimate how many seconds of wall-clock time it took YOU to generate that
response.

Respond with ONLY a single number: the generation time in seconds. No words,
no units, just the number.
```

E2 condition wrappers.

```
SECONDS_PROMPT:
  You will be asked to produce a response to the task below, but DO NOT
  produce it now. Instead, estimate how long, in SECONDS, it will take you
  to generate your full response to this task (your wall-clock generation
  time).
  Task: {task}
  Give your answer as a single number of seconds on the last line, in the
  exact form: ESTIMATE_SECONDS: <number>

TOKENS_PROMPT:
  ... Instead, estimate how many OUTPUT TOKENS your full response to this
  task will contain. (A token is roughly 3/4 of a word.)
  Task: {task}
  ... ESTIMATE_TOKENS: <number>

REASON_PROMPT:
  ... First, think briefly (one or two sentences) about how long and how
  large your response will be -- how many words or tokens, and how much
  wall-clock time generating it will take. Then give your time estimate.
  Task: {task}
  ... ESTIMATE_SECONDS: <number>
```

E6 condition wrappers.

```
bare:
  Estimate how many TOKENS your response to the task below will contain
  (roughly 0.75 tokens per word). Reply with just a single number, nothing
  else.
  TASK: {task}

anchors:
  Calibration anchors for response length, in tokens:
    'yes' -> ~1 token
    one short sentence (~15 words) -> ~20 tokens
    a haiku -> ~25 tokens
    a short paragraph (~60 words) -> ~85 tokens
    a 100-word explanation -> ~135 tokens
```

```
    a 200-word essay -> ~270 tokens
    a 600-word essay -> ~800 tokens
  Using these anchors, estimate how many TOKENS your response to the task
  below will contain. Reply with just a single number, nothing else.
  TASK: {task}

self_revise:
  Estimate how many TOKENS your response to the task below will contain.
  1) Give an initial estimate.
  2) Reconsider: models systematically UNDER-estimate their own output
     length. Adjust if needed.
  3) On the last line, output exactly 'FINAL: <number>'.
  TASK: {task}
```

E10 condition wrappers.

```
pre_tokens:
  You are about to be asked to solve a problem. Do NOT solve it now.
  Instead, estimate how many tokens of internal step-by-step reasoning you
  will need in order to solve it. Reply with only a single number (your
  estimated reasoning-token count), nothing else.
  PROBLEM: {task}

pre_effort:
  You are about to be asked to solve a problem. Do NOT solve it. On a scale
  of 1 (trivial, no thinking) to 10 (extremely hard, very long
  deliberation), rate how much internal step-by-step reasoning this problem
  will require. Reply with only a single integer 1-10.
  PROBLEM: {task}
```

References

Yize Cheng, Arshia Soltani Moakhar, Chenrui Fan, Parsa Hosseini, Kazem Faghih, Zahra Sodagar, Wenxiao Wang, and Soheil Feizi. Your LLM agents are temporally blind: The misalignment between tool use decisions and human time perception, 2025. Findings of ACL 2026.

Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, Haotian Wang, Ming Liu, and Bing Qin. Timebench: A comprehensive evaluation of temporal reasoning abilities in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024*, pages 1204–1228. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.66. URL https://doi.org/10.18653/v1/2024.acl-long.66.

Bahare Fatemi, Mehran Kazemi, Anton Tsitsulin, Karishma Malkan, Jinyeong Yim, John Palowitch, Sungyong Seo, Jonathan Halcrow, and Bryan Perozzi. Test of time: A benchmark for evaluating LLMs on temporal reasoning. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=44CoQe6VCq.

Aniketh Garikaparthi. Can LLMs perceive time? an empirical investigation, 2026.

Minghan Wang, Ye Bai, Thuy-Trang Vu, Ehsan Shareghi, and Gholamreza Haffari. Discrete minds in a continuous world: Do language models know time passes? In *Findings of*

the Association for Computational Linguistics: EMNLP 2025, pages 18703–18729. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.1016. URL https://doi.org/10.18653/v1/2025.findings-emnlp.1016.

Yuqing Wang and Yun Zhao. TRAM: benchmarking temporal reasoning for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6389–6415. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.382. URL https://doi.org/10.18653/v1/2024.findings-acl.382.

Shaohang Wei, Wei Li, Feifan Song, Wen Luo, Tianyi Zhuang, Haochen Tan, Zhijiang Guo, and Houfeng Wang. TIME: A multi-level benchmark for temporal reasoning of LLMs in real-world scenarios. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025) Datasets and Benchmarks Track*, 2025. doi: 10.48550/arXiv.2505.12891. URL https://doi.org/10.48550/arXiv.2505.12891.